

Arquitectura Orientada a Microservicios

Sebastián Vallejo

Ingeniería de Sistemas Mod. Virtual, Uniremington

Lenguaje de Programación Avanzado 2

Nixon Duarte Acosta

2026

Arquitectura Orientada a Microservicios

El presente trabajo aborda los fundamentos de la arquitectura orientada a microservicios, sus diferencias frente a la arquitectura monolítica y su relevancia en la construcción de aplicaciones modernas. Su propósito es reconocer los principios básicos de este estilo arquitectónico, identificar sus ventajas, desventajas y escenarios de aplicación, y experimentar con un ejemplo práctico de comunicación entre servicios.

Para la actividad práctica se construyeron dos microservicios independientes: un Servicio de Usuarios, desarrollado en Python 3.13 con FastAPI, y un Servicio de Pedidos, desarrollado en TypeScript sobre Bun 1.4. Cada servicio cuenta con su propia base de datos PostgreSQL, ambos se comunican mediante una API REST y el conjunto se ejecuta con Docker Compose.

El documento se organiza en cinco secciones, cada una iniciada en una página nueva. La primera presenta el cuadro comparativo entre la arquitectura monolítica y la de microservicios. La segunda contiene el ensayo breve sobre las razones por las que las grandes empresas tecnológicas adoptan microservicios, que ocupa una única página. La tercera documenta el diseño mediante diagramas de arquitectura; la cuarta reúne la evidencia del código y de su ejecución, y la quinta incluye el enlace al video de sustentación. Al final se presenta la lista de referencias.

Cuadro Comparativo

La Tabla 1 compara la arquitectura monolítica y la arquitectura orientada a microservicios a partir de diez criterios, el doble del mínimo solicitado. Además de describir cada enfoque, la última columna vincula cada criterio con un archivo o comando del repositorio del ejercicio práctico, de modo que la diferencia pueda verificarse al ejecutar el código.

Interpretación de la Comparación

Las diez filas se reducen a un solo intercambio: los microservicios cambian complejidad interna por complejidad operativa.

Un monolito concentra la dificultad dentro del código: a medida que crece, los módulos se enredan y nadie se atreve a tocar el núcleo. Los microservicios extraen esa dificultad del código y la depositan en la infraestructura: el código de cada servicio queda pequeño y comprensible, pero aparecen problemas que antes no existían —latencia, fallos parciales, versionado de contratos, consistencia eventual.

El cambio conviene cuando la complejidad organizativa ya supera a la técnica: cuando el cuello de botella no es escribir el código, sino coordinar a la gente que lo escribe. Por eso es una arquitectura que rinde en empresas grandes y suele castigar a los equipos pequeños, que pagan el costo operativo sin tener el problema de coordinación que lo justifica.

Este repositorio hace visible ese precio: para dos servicios que en un monolito serían dos módulos y una llamada a función, aquí hacen falta cuatro contenedores, tres redes, dos esquemas de base de datos, un adaptador HTTP con timeout y reintento, y una política explícita de qué hacer cuando el otro servicio no responde.

Tabla 1*Comparación Entre la Arquitectura Monolítica y la Arquitectura de Microservicios*

Criterio	Arquitectura monolítica	Arquitectura de microservicios	Evidencia en el repositorio
Unidad de despliegue	Un solo artefacto. Cambiar una línea en el módulo de pedidos obliga a desplegar también usuarios, facturación y todo lo demás.	Cada servicio se despliega por separado y en su propio ritmo.	<code>make up-users</code> levanta Usuarios sin tocar Pedidos. Son dos imágenes Docker independientes, con dos <code>Dockerfile</code> distintos.
Escalabilidad	Se replica la aplicación completa, aunque el cuello de botella sea un solo módulo. Se paga por escalar código que no lo necesita.	Se replica solo el servicio saturado. El escalado sigue a la demanda real.	Cada servicio es una imagen con su propio proceso: replicar Pedidos (<code>--scale orders-service=3</code> , quitando antes el puerto fijo del host) no obliga a replicar Usuarios.
Base de datos	Una base compartida. La integridad referencial la garantiza el motor con claves foráneas y transacciones ACID.	Una base por servicio (<i>database per service</i>). Ningún servicio lee las tablas de otro.	<code>orders.user_id</code> no tiene FOREIGN KEY hacia <code>users</code> (<code>services/orders-service/db/schema.sql</code>): la tabla vive en otro contenedor, en otra red.
Acoplamiento e integridad	Acoplamiento en tiempo de compilación: llamada a función, con el compilador como red de seguridad.	Acoplamiento por contrato de red: la validación se sostiene llamando al otro servicio.	<code>CreateOrder</code> consulta el puerto <code>UserDirectory</code> antes de insertar (<code>services/orders-service/src/application/create-order.ts</code>).
Stack tecnológico	Uno solo para todo el sistema. Cambiar de lenguaje o de versión es una migración global.	Cada servicio elige el suyo según el problema que resuelve.	Usuarios en Python 3.13 + FastAPI; Pedidos en TypeScript + Bun 1.4. Conviven sin conocerse: solo comparten JSON sobre HTTP.
Aislamiento de fallos	Un fallo grave (fuga de memoria, excepción no controlada) derriba todo el proceso, incluidas las funciones sanas.	El fallo queda confinado al servicio afectado; el resto sigue operando, quizá degradado.	<code>make demo-fallo</code> : con Usuarios apagado, <code>POST /orders</code> responde 503, pero <code>GET /orders</code> sigue devolviendo 200.

Criterio	Arquitectura monolítica	Arquitectura de microservicios	Evidencia en el repositorio
Organización de los equipos (Ley de Conway)	Muchos equipos sobre un mismo código: conflictos de integración y una cola única de release.	Cada equipo es dueño de su servicio, de su base y de su despliegue.	Los dos servicios no comparten ni una línea de código: <code>services/users-service</code> y <code>services/orders-service</code> son mundos separados.
Complejidad operativa	Baja. Un proceso, un log, un despliegue; se depura con un depurador local.	Alta. Red, descubrimiento de servicios, trazas distribuidas, versionado de APIs, orquestación.	Para correr esto hacen falta 4 contenedores, 3 redes y 2 volúmenes (<code>compose.yaml</code>) donde un monolito usaría 1 proceso y 1 base. Ese es el precio.
Consistencia transaccional	<code>BEGIN ... COMMIT</code> abarca toda la operación de negocio, aunque toque varios módulos.	No hay transacción distribuida. Hay consistencia eventual, <i>sagas</i> y compensaciones.	Si Usuarios se cayera justo después de validar y antes de insertar, no habría <i>rollback</i> automático entre servicios: por eso <code>CreateOrder</code> valida antes de persistir.
Estrategia de pruebas	Las pruebas de integración son baratas: todo está en el mismo proceso.	Las unitarias siguen siendo baratas, pero probar la integración real exige levantar varios servicios o sustituirlos por dobles.	<code>make test</code> : 37 pruebas en milisegundos usando dobles (<code>tests/doubles.ts</code> , <code>tests/in_memory_user_repository.py</code>). Probar el sistema completo requiere <code>make up</code> .

Nota. La última columna remite a archivos y comandos del repositorio del ejercicio práctico. Elaboración propia con base en Newman (2021) y Fowler (2015).

Ensayo Breve: ¿Por Qué las Grandes Empresas Tecnológicas Adoptan Microservicios?

Las grandes empresas tecnológicas no adoptan microservicios porque sean técnicamente superiores, sino porque su sistema monolítico dejó de ser un problema de software para convertirse en uno de organización. Conway (1968) observó que los sistemas reproducen la estructura de comunicación de quienes los diseñan: cuando miles de ingenieros comparten un único artefacto desplegable, el código no se vuelve más lento de escribir, sino de coordinar. Amazon enfrentó ese problema hacia 2002, cuando exigió que todos sus equipos se comunicaran únicamente mediante interfaces de servicio (Yegge, 2011).

El segundo motivo es la frecuencia de despliegue. En un monolito, corregir un módulo obliga a desplegar la aplicación completa, lo que empuja a publicar poco y en lotes riesgosos; al reducir el alcance de cada cambio, Netflix pudo migrar su plataforma a un conjunto de servicios que evolucionan de manera independiente (Mauro, 2015). A ello se suman el escalado selectivo, que evita replicar componentes que no están bajo carga; el aislamiento de fallos, que permite degradar una función sin detener el sistema, y la libertad de elegir la tecnología adecuada para cada servicio (Newman, 2021).

Sin embargo, el costo es real. Una llamada a función se convierte en una llamada de red, con latencia y fallos parciales, y la transacción ACID cede su lugar a la consistencia eventual. Por ello, Fowler (2015) recomienda comenzar con un monolito bien modularizado y extraer servicios solo cuando el problema lo exija. El propio equipo de Prime Video reunió en una sola aplicación un servicio de monitoreo que había construido de forma distribuida, y con ello redujo sus costos de infraestructura en más de un 90 % (Kolny, 2023).

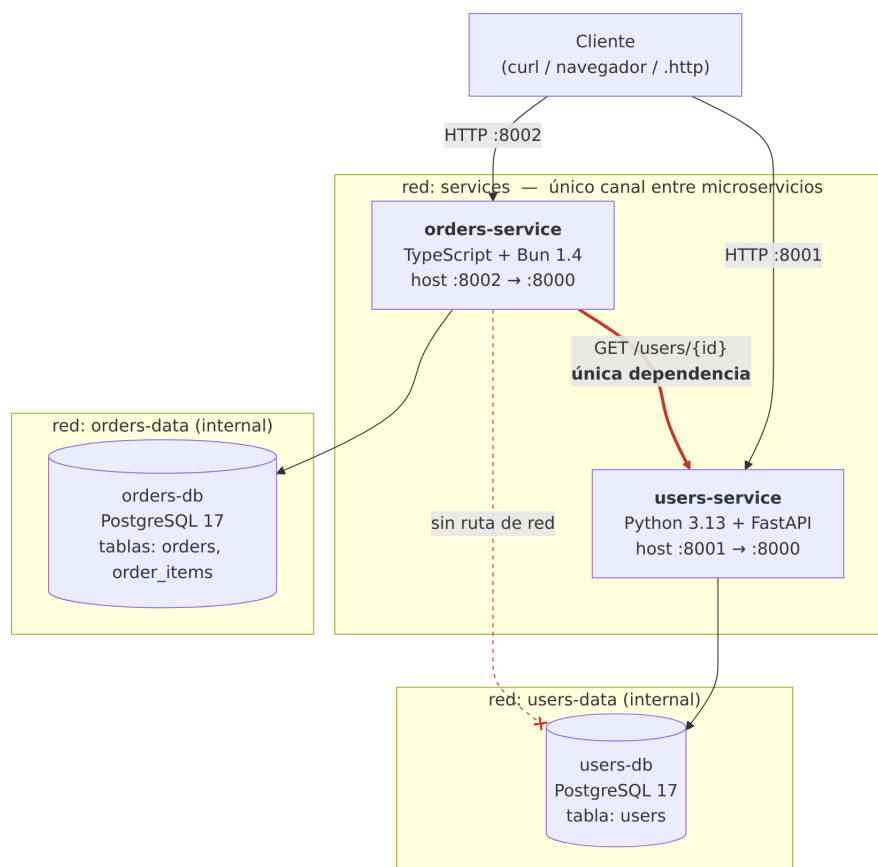
En conclusión, los microservicios resuelven un problema específico: permitir que muchos equipos avancen en paralelo sobre un sistema de gran tamaño. Las empresas que los adoptan con éxito ya enfrentaban ese problema; adoptarlos sin tenerlo implica asumir toda su complejidad operativa sin obtener el beneficio que la justifica.

Diagrama de Arquitectura

El diseño del ejercicio práctico se documenta mediante tres diagramas complementarios. La Figura 1 muestra los contenedores que se ejecutan y las redes que los conectan; la Figura 2 describe la secuencia de la única operación que requiere comunicación entre servicios, y la Figura 3 presenta la organización interna de cada servicio de acuerdo con los principios de la arquitectura limpia (Martin, 2017).

Diagrama de Contenedores

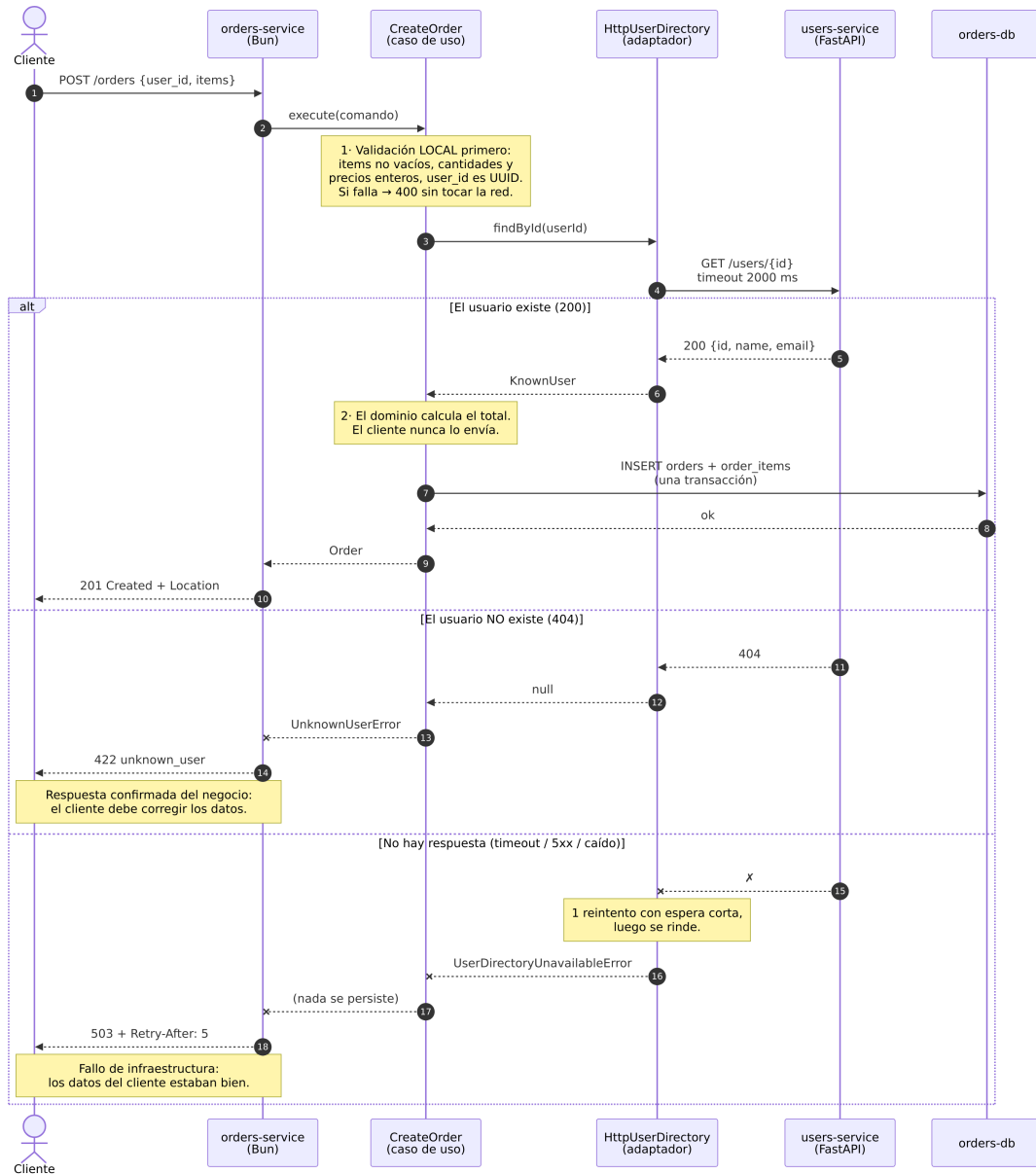
El sistema se compone de cuatro contenedores distribuidos en tres redes de Docker. Cada microservicio accede únicamente a su propia base de datos, y la red compartida `services` es el único canal entre ambos, de modo que la comunicación ocurre exclusivamente por HTTP. Como se observa en la Figura 1, el Servicio de Pedidos no tiene ruta de red hacia la base de datos del Servicio de Usuarios: el aislamiento de los datos se impone en la infraestructura y no depende de la disciplina del código.

Figura 1*Contenedores, Redes y Dependencias del Sistema*

Nota. La línea continua roja representa la única dependencia entre microservicios, la consulta `GET /users/{id}`. La línea discontinua terminada en cruz representa una conexión inexistente: `orders-service` no pertenece a la red `users-data`.

Diagrama de Secuencia de la Creación de un Pedido

La Figura 2 detalla la operación `POST /orders`, en la que el Servicio de Pedidos consulta al Servicio de Usuarios antes de registrar un pedido. La secuencia contempla tres resultados posibles: el usuario existe, el usuario no existe o el Servicio de Usuarios no responde.

Figura 2*Secuencia de la Operación de Creación de un Pedido*

Nota. El Servicio de Pedidos valida localmente el pedido antes de consultar al Servicio de Usuarios, con un tiempo máximo de espera de 2 s y un único reintento. Si no obtiene respuesta, no persiste el pedido.

La distinción entre los dos últimos resultados constituye la decisión de diseño central del servicio y se resume en la Tabla 2. Cuando el Servicio de Usuarios responde con el código 404

existe certeza de que el usuario no está registrado; cuando no responde, no es posible saberlo.

Tratar ambos casos como un mismo error provocaría que el sistema rechazara pedidos de usuarios válidos cada vez que la red fallara, atribuyendo al cliente un fallo propio.

Tabla 2

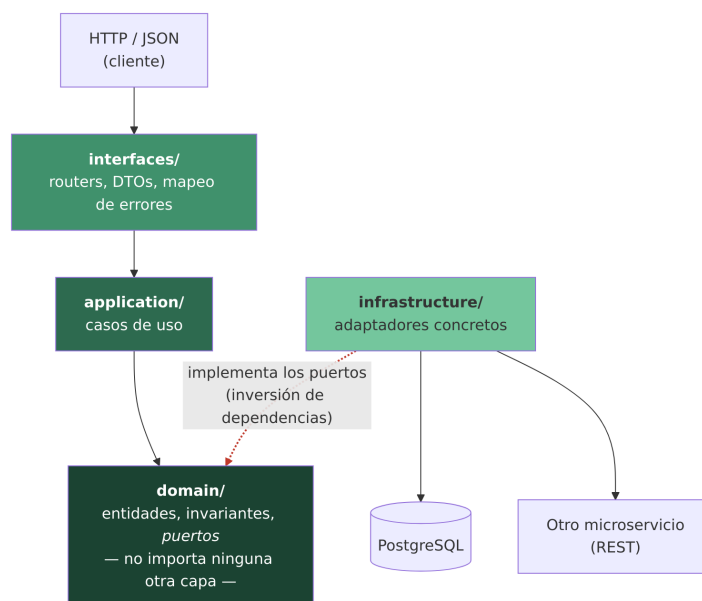
Tratamiento de las Respuestas del Servicio de Usuarios

Situación	Conocimiento del sistema	Error de dominio	Código HTTP	Acción del cliente
El Servicio de Usuarios responde 404	El usuario no existe	<code>UnknownUserError</code>	422	Corregir el identificador del usuario
El Servicio de Usuarios no responde	Se desconoce si el usuario existe	<code>UserDirectoryUnavailableError</code>	503	Reintentar la operación más tarde

Nota. Los errores de dominio se definen en `services/orders-service/src/domain/errors.ts`.

Capas de la Arquitectura Limpia

Ambos servicios comparten la misma organización interna en cuatro capas, aunque estén escritos en lenguajes distintos, como muestra la Figura 3. La regla de dependencias establece que estas apuntan siempre hacia el dominio: la capa de infraestructura implementa las interfaces que el dominio declara, y nunca a la inversa (Martin, 2017).

Figura 3*Capas de la Arquitectura Limpia y Regla de Dependencias*

Nota. La flecha discontinua indica que la infraestructura implementa los puertos declarados por el dominio, es decir, la inversión de dependencias.

Un puerto es una interfaz declarada por el dominio, y un adaptador es su implementación concreta, que se elige al iniciar el servicio. La Tabla 3 relaciona los puertos de cada servicio con sus adaptadores. Disponer de adaptadores de prueba permite verificar el dominio y los casos de uso sin base de datos ni red, incluido el escenario en que el Servicio de Usuarios no está disponible.

Tabla 3*Puertos y Adaptadores de los Microservicios*

Servicio	Puerto	Adaptador de producción	Adaptadores de prueba
Usuarios	UserRepository	PostgresUserRepository	InMemoryUserRepository
Pedidos	OrderRepository	PostgresOrderRepository	InMemoryOrderRepository
Pedidos	UserDirectory	HttpUserDirectory	FakeUserDirectory, EmptyUserDirectory, BrokenUserDirectory

Nota. Los adaptadores de prueba se encuentran en la carpeta `tests/` de cada servicio.

Evidencia del Código

El código fuente completo del ejercicio práctico, junto con su historial de commits, está publicado en un repositorio público de GitHub disponible en <https://github.com/svallejo-dev/lpa-two-microservices>. La Tabla 4 describe su contenido. El único requisito para ejecutarlo es Docker: los comandos `make up`, `make test` y `make demo` levantan el sistema, ejecutan las pruebas automatizadas y recorren el flujo completo, respectivamente.

Tabla 4

Contenido del Repositorio

Ruta	Contenido
<code>services/users-service/</code>	Servicio de Usuarios: Python 3.13, FastAPI y PostgreSQL
<code>services/orders-service/</code>	Servicio de Pedidos: TypeScript, Bun 1.4 y PostgreSQL
<code>services/orders-service/openapi.yaml</code>	Contrato OpenAPI 3.1 del Servicio de Pedidos, redactado manualmente y verificado por pruebas
<code>compose.yaml</code>	Definición de los cuatro contenedores, las tres redes y los dos volúmenes
<code>Makefile</code>	Comandos para levantar, probar y demostrar el sistema
<code>docs/</code>	Cuadro comparativo, ensayo, arquitectura y contrato de la API
<code>scripts/</code>	Guiones de demostración y generador de este documento
<code>entrega/</code>	Este documento en formato PDF

Nota. Las rutas son relativas a la raíz del repositorio.

Comunicación Entre los Servicios

La Figura 4 presenta el caso de uso `CreateOrder`, el único punto en el que un microservicio requiere información que pertenece al otro. El caso de uso depende de la interfaz `UserDirectory` y no de un cliente HTTP concreto, lo que permite probarlo sin red.

Figura 4

Caso de Uso de Creación de un Pedido

```
async execute(command: CreateOrderCommand): Promise<Order> {
  const order = Order.place(command.userId, command.items);

  const user = await this.users.findById(order.userId);
  if (user === null) {
    throw new UnknownUserError(order.userId);
  }

  await this.orders.add(order);
  return order;
}
```

Nota. Fragmento del archivo `services/orders-service/src/application/create-order.ts`. El método ejecuta tres pasos: valida el pedido localmente, de modo que uno mal formado se rechaza sin gastar una llamada de red; consulta al Servicio de Usuarios a través del puerto `UserDirectory`, que lanza una excepción si no obtiene respuesta; y solo entonces lo persiste.

Resultados de la Ejecución

Las salidas que se presentan a continuación se capturaron del sistema en funcionamiento. La Figura 5 muestra los cuatro contenedores en estado saludable y la Figura 6, el resultado de las pruebas automatizadas. La Figura 7 recorre el flujo completo entre ambos microservicios, y la Figura 8 muestra el comportamiento del sistema cuando el Servicio de Usuarios deja de responder.

Figura 5

Estado de los Contenedores en Ejecución

SERVICE	STATUS
orders-db	Up 40 minutes (healthy)
orders-service	Up 40 minutes (healthy)
users-db	Up 40 minutes (healthy)
users-service	Up 40 minutes (healthy)

Nota. Salida del comando `docker compose ps`.

Figura 6*Resultado de las Pruebas Automatizadas*

```
==> Pruebas: Servicio de Usuarios  
..... [100%]  
==> Pruebas: Servicio de Pedidos  
21 pass  
0 fail
```

Nota. Salida del comando `make test`. Cada punto de la segunda línea corresponde a una prueba superada del Servicio de Usuarios, 16 en total; el Servicio de Pedidos reporta 21 pruebas superadas. Ninguna prueba requiere base de datos ni red.

Figura 7

Flujo Completo Entre los Dos Microservicios

```

0. Estado de las dos dependencias del Servicio de Pedidos
GET http://localhost:8002/health/ready
{"status":"ready","service":"orders-service","dependencies":
{"database":"ok","users_service":"ok"}}
[HTTP 200]

1. Crear un usuario en el Servicio de Usuarios (Python + FastAPI)
{"id":"7ca2edfd-8f94-44e0-993f-40bdc9ced8a1","name":"Estudiante
LPA2","email":"demo-1789103458@example.com","created_at":"2026-09-11T05:10:58.360924Z"}
id del usuario: 7ca2edfd-8f94-44e0-993f-40bdc9ced8a1

2. Crear un pedido en el Servicio de Pedidos (TypeScript + Bun)
Pedidos NO tiene la tabla de usuarios: consulta a Usuarios por HTTP antes de
insertar.
POST http://localhost:8002/orders
{"id":"6a42510c-1199-4f39-a9ce-93d979490b0e","user_id":"7ca2edfd-8f94-44e0-993f-40bdc9ced8a1","status":"PENDIENTE",
"total":54000,"items":[{"sku":"TECLADO","quantity":2,"unit_price":15000,"subtotal":
30000},{ "sku":"MOUSE","quantity":3,"unit_price":8000,"subtotal":
24000}], "created_at":"2026-09-11T05:10:58.396Z"}
[HTTP 201]
El total (54000) lo calculo el dominio; el cliente nunca lo envio.

3. Consultar los pedidos de ese usuario
GET http://localhost:8002/orders?user_id=7ca2edfd-8f94-44e0-993f-40bdc9ced8a1
[{"id":"6a42510c-1199-4f39-a9ce-93d979490b0e","user_id":"7ca2edfd-8f94-44e0-993f-40bdc9ced8a1","status":"PENDIENTE",
"total":54000,"items":[{"sku":"TECLADO","quantity":2,"unit_price":15000,"subtotal":
30000},{ "sku":"MOUSE","quantity":3,"unit_price":8000,"subtotal":
24000}], "created_at":"2026-09-11T05:10:58.396Z"}]
[HTTP 200]

4. Intentar un pedido para un usuario que no existe
Usuarios responde 404 -> Pedidos lo traduce a 422: los datos del cliente estan mal.
POST http://localhost:8002/orders
{"error":"unknown_user","message":"El usuario '00000000-0000-0000-0000-000000000000'
no existe en el Servicio de Usuarios."}
[HTTP 422]

5. Intentar un pedido invalido (sin lineas)
Se rechaza en el dominio, sin gastar una llamada de red al otro servicio.
POST http://localhost:8002/orders
{"error":"empty_order","message":"Un pedido debe tener al menos una linea."}
[HTTP 400]

Listo.
Para ver que ocurre cuando el Servicio de Usuarios se cae: make demo-fallo

```

Nota. Salida del comando `make demo`. Se observan las respuestas 201 al crear el pedido, 422 ante un usuario inexistente y 400 ante un pedido sin líneas.

Figura 8

Comportamiento del Sistema con el Servicio de Usuarios Detenido

```

0. Preparacion: un usuario que SI existe
   id: 255e57c5-9dd2-4def-8743-cafed30aed1c (este usuario es real, el pedido deberia
   poder crearse)

1. Apagando el Servicio de Usuarios...
   Servicio de Usuarios DETENIDO.

2. Intentando crear un pedido de ese mismo usuario valido
   HTTP/1.1 503 Service Unavailable
   retry-after: 5
   {"error":"user_directory_unavailable","message":"No se pudo verificar el usuario
   porque el Servicio de Usuarios no respondio (The operation timed out.). El pedido no
   fue creado; reintente mas tarde."}
   Tardo 5 s: el timeout acota la espera en lugar de colgar la peticion.
   503, no 400: el problema no son los datos del cliente. Y el pedido NO se guardo.

3. Lo que SIGUE funcionando mientras Usuarios esta caido
   GET /orders -> [HTTP 200]
   Las lecturas de pedidos no dependen de Usuarios: el fallo esta acotado, no tumba el
   servicio entero.

4. Diagnostico automatico
   {"status":"degraded","service":"orders-service","dependencies":
   {"database":"ok","users_service":"unreachable"}}
   La sonda dice exactamente cual de las dos dependencias fallo.

5. Restaurando el Servicio de Usuarios...
   Servicio de Usuarios ARRIBA.

6. El mismo pedido, ahora sin cambiar nada en el cliente
   {"id":"12d2d661-460a-4ba7-aa56-c6a7d99600f9","user_id":"255e57c5-9dd2-4def-8743-
   cafed30aed1c","status":"PENDIENTE","total":250000,"items":[{"sku":"SSD","quantity":
   1,"unit_price":250000,"subtotal":250000}],"created_at":"2026-09-11T05:11:06.800Z"}
   [HTTP 201]
   Recuperacion automatica: no hubo que reiniciar Pedidos ni limpiar datos a medias.

```

Nota. Salida del comando `make demo-fallo`. La creación del pedido responde 503 sin persistir datos, la consulta de pedidos continúa respondiendo 200 y, al restaurar el Servicio de Usuarios, la misma operación vuelve a responder 201.

Video de Sustentación

La sustentación recorre el sistema en funcionamiento siguiendo el mismo guion de los comandos `make demo` y `make demo-fallo`: la creación de un usuario, la creación de un pedido con la validación entre servicios, los errores de negocio y el comportamiento del sistema cuando el Servicio de Usuarios deja de responder. La grabación está disponible en el siguiente enlace:

<https://youtu.be/t6DTXp7V4aU>

El código fuente al que se refiere la sustentación, junto con este documento y su historial de commits, está disponible en el repositorio del proyecto:

<https://github.com/svallejo-dev/lpa-two-microservices>

Referencias

- Conway, M. E. (1968). How do committees invent? *Datamation*, 14(4), 28–31.
- Fowler, M. (2015, 3 de junio). MonolithFirst. *Martin Fowler*. <https://martinfowler.com/bliki/MonolithFirst.html>
- Kolny, M. (2023, 22 de marzo). Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%. *Prime Video Tech*. <https://web.archive.org/web/20230323220106/https://www.primevideotech.com/video-streaming/scaling-up-the-prime-video-audio-video-monitoring-service-and-reducing-costs-by-90>
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- Mauro, T. (2015, 19 de febrero). Adopting microservices at Netflix: Lessons for architectural design. *NGINX Blog*. <https://web.archive.org/web/20160114045720/https://www.nginx.com/blog/microservices-at-netflix-architectural-best-practices/>
- Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.^a ed.). O'Reilly Media.
- Yegge, S. (2011, 12 de octubre). *Stevey's Google platforms rant*. GitHub Gist. <https://gist.github.com/chitchcock/1281611>